

HW1

Vinisha Bala Dhayanidhi

October 8, 2026

Q1: readme

Files. `q1.py` (Python program), `make_voc.sh` (two-line shell wrapper), `hw1.cmd` (condor file), and `data.out` (output from running the job on `patas` on `$hwDir/examples/data.in`).

How it works. `q1.py` reads the input and output filenames from `sys.argv`, reads the input file line by line, splits each line into tokens with `line.split()`, and counts them in a dictionary. It then writes one `token<TAB>frequency` line per token to the output file. The shell script is:

```
#!/bin/sh
python3 q1.py "$1" "$2"
```

Problems I ran into and how I fixed them.

1. **Tokenizing with `split(" ")` vs. `split()`.** I tested both on `"a b\tc\n"`. `split(" ")` gave `['a', ' ', 'b\tc\n']` (an empty string for the double space, and the tab and newline stayed inside a token), while `split()` gave `['a', 'b', 'c']`. The assignment instructions ask to split “by whitespace”, so I used `split()`.
2. **Wrong separator in the output.** My first versions wrote a single space, then four spaces, between the token and the count, but the assignment requires a tab. `cat -A tmp.out` showed no `^I` characters, which is how a tab appears. I fixed it by writing the tab escape explicitly: `fout.write(f"{token}\t{freq}\n")`. After that, `cat -A` showed `the^I10$`.
3. **`FileNotFoundError: '/examples/sample.in'`.** The shell variable `$hwDir` was empty (it is not set in a new session), so `$hwDir/examples/sample.in` expanded to `/examples/sample.in`. The Python code was correct; the problem was in the shell. I fixed it with `export hwDir=/mnt/dropbox/26-2` and added that line to `~/.bashrc` so it is set at every login. I hit this error twice, once per new session, before adding it to `.bashrc`.
4. **`diff` against `sample.out` showed differences.** The same lines appeared on both sides in different positions, so the content matched but the sort order differed (my `sort` used ASCII order, while the gold file seems to be ordered differently). I compared them fairly by sorting both files with the same command and then diffing:

```
sort ./tmp.out > a.txt
sort $hwDir/examples/sample.out > b.txt
diff a.txt b.txt
```

`diff` printed nothing, so the outputs are identical. I also noticed that the spec's command `sort ./tmp.out ./tmp.out2` would treat both names as input files, so I used `>` to redirect the sorted output instead.

Testing. Running `./make_voc.sh $hwDir/examples/sample.in ./tmp.out`, sorting the result, and diffing it against the sorted `sample.out` gave no differences. `hw1.cmd` uses `executable=./make_voc.sh` and `arguments="/mnt/dropbox/26-27/570/hw1/examples/data.in ./data.out"`. The tarball was created with `tar -cvzf hw.tar.gz *`, and `tar -tzf` lists `data.out`, `hw1.cmd`, `make_voc.sh`, `q1.py`, and the condor log files.

Q2: At least 4 heads in 5 flips ($p = 0.8$)

“At least 4” means exactly 4 heads or exactly 5 heads. These events are mutually exclusive, so their probabilities can be added. The number of heads in n independent flips is $\text{Binomial}(n, p)$:

$$P(k \text{ heads}) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Therefore

$$\begin{aligned} P(\text{at least 4 heads}) &= \binom{5}{4} (0.8)^4 (0.2)^1 + \binom{5}{5} (0.8)^5 (0.2)^0 \\ &= 5(0.4096)(0.2) + 1(0.32768) \\ &= 0.4096 + 0.32768 = \mathbf{0.73728}. \end{aligned}$$

The factor $\binom{5}{4} = 5$ counts the positions the single tail can occupy (HHHHT, HHHTH, HHTHH, HTHHH, THHHH).

Q3: Joint distribution of X and Y

(a) $P(X)$. Marginalize over Y by summing each column:

$$P(X = 0) = 0.50 + 0.10 = 0.60, \quad P(X = 1) = 0.25 + 0.15 = 0.40.$$

These sum to 1.

(b) $P(Y)$. Marginalize over X by summing each row:

$$P(Y = 0) = 0.50 + 0.25 = 0.75, \quad P(Y = 1) = 0.10 + 0.15 = 0.25.$$

These sum to 1.

(c) $P(Y | X)$. Use $P(Y | X) = P(X, Y)/P(X)$ for each cell:

	$X = 0$	$X = 1$
$P(Y = 0 X)$	$0.50/0.60 = 5/6 \approx 0.833$	$0.25/0.40 = 0.625$
$P(Y = 1 X)$	$0.10/0.60 = 1/6 \approx 0.167$	$0.15/0.40 = 0.375$

Each column sums to 1, as a conditional distribution must.

(d) **Independence.** X and Y are independent iff $P(X, Y) = P(X)P(Y)$ for every cell. Check one:

$$P(X = 0)P(Y = 0) = 0.60 \times 0.75 = 0.45 \neq 0.50 = P(X = 0, Y = 0).$$

So X and Y are **not** independent. Equivalently, $P(Y = 0 \mid X = 0) = 0.833 \neq P(Y = 0) = 0.75$, so knowing X changes our belief about Y .

Q4: Three coins

Let C be the chosen coin and H the event of getting a head. $P(c_1) = 0.2$, $P(c_2) = 0.5$, $P(c_3) = 0.3$ and $P(H \mid c_1) = 0.1$, $P(H \mid c_2) = 0.4$, $P(H \mid c_3) = 0.7$.

(a) By the law of total probability:

$$\begin{aligned} P(H) &= \sum_i P(H \mid c_i)P(c_i) \\ &= (0.1)(0.2) + (0.4)(0.5) + (0.3)(0.7) \\ &= 0.02 + 0.20 + 0.21 = \mathbf{0.43}. \end{aligned}$$

(b) By Bayes' rule:

$$P(c_1 \mid H) = \frac{P(H \mid c_1)P(c_1)}{P(H)} = \frac{0.1 \times 0.2}{0.43} = \frac{0.02}{0.43} = \frac{2}{43} \approx \mathbf{0.0465}.$$

The denominator is exactly the answer to part (a).

Q5: Independence and conditional independence

(a) **No.** Conditional independence given Z does not imply independence.

Counterexample. Let $Z \in \{1, 2\}$ pick a coin, each with probability 0.5. Coin 1 has $P(\text{head}) = 0.1$ and coin 2 has $P(\text{head}) = 0.9$. Given the coin, X and Y are two independent flips of that coin ($\text{head} = 1$). So $X \perp Y \mid Z$ by construction. But

$$P(X = 1) = P(Y = 1) = 0.5(0.1) + 0.5(0.9) = 0.5,$$

$$P(X = 1, Y = 1) = 0.5(0.1)^2 + 0.5(0.9)^2 = 0.005 + 0.405 = 0.41 \neq 0.25 = P(X = 1)P(Y = 1).$$

So X and Y are not independent. Intuitively, Z is a common cause: seeing $X = 1$ makes the coin probably the heads-heavy one, which raises the chance that $Y = 1$.

(b) **No.** Independence does not imply conditional independence.

Counterexample. Let X, Y be independent fair bits and let $Z = X \oplus Y$ (XOR). Then $P(X = 1, Y = 1) = 0.25 = P(X = 1)P(Y = 1)$, so $X \perp Y$. But given $Z = 0$ we must have $X = Y$, so

$$P(X = 1, Y = 1 \mid Z = 0) = \frac{0.25}{0.5} = 0.5,$$

while $P(X = 1 \mid Z = 0)P(Y = 1 \mid Z = 0) = 0.5 \times 0.5 = 0.25$. Since $0.5 \neq 0.25$, X and Y are not conditionally independent given Z . Here Z is a common effect, and conditioning on it creates a dependence. This matches the lecture example: burglary and earthquake are independent, but both can set off the alarm. Once we know the alarm went off, learning that there was an earthquake makes a burglary less likely.

Q6: Formal grammars

(a) **Regular grammar for a^*cb^* .** A regular (right-linear) grammar has rules $A \rightarrow xB$ or $A \rightarrow x$ with x a terminal:

$$\begin{aligned} S &\rightarrow aS \mid c \mid cB \\ B &\rightarrow bB \mid b \end{aligned}$$

S generates any number of a 's, then the single c . After the c we may stop (rule $S \rightarrow c$) or move to B , which generates one or more b 's.

(b) **CFG for $\{a^n cb^n \mid n \geq 0\}$.**

$$S \rightarrow aSb \mid c$$

Each use of $S \rightarrow aSb$ adds one a on the left and one b on the right, so the counts stay equal. $S \rightarrow c$ ends the recursion in the middle and using it immediately gives $n = 0$.

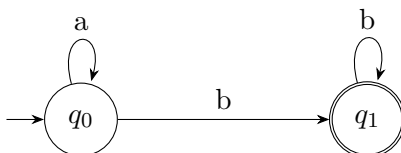
(c) **CFG for $\{xy \mid x \in \{a, b, c\}^*, y = x^R\}$.**

$$S \rightarrow aSa \mid bSb \mid cSc \mid \varepsilon$$

Each rule wraps the inside string with a matching symbol on both ends, so the second half is the mirror image of the first. $S \rightarrow \varepsilon$ gives the empty string ($x = \varepsilon$).

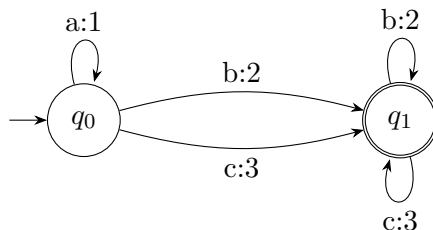
Q7: FSA and FST

(a) **FSA for a^*b^+ .**



q_0 means “no b seen yet” and q_1 means “at least one b seen”. After the first b , an a has no transition, so aba and bab are rejected; aaa never reaches q_1 , so it is rejected too.

(b) **FST.** Arc labels are **input:output**.



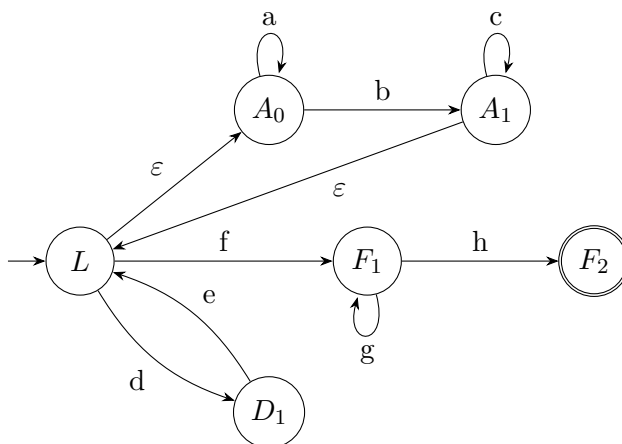
It is the same skeleton as (a), with c allowed as well and each arc emitting the digit. For $aabcbbc$ the path is $q_0 \xrightarrow{a:1} q_0 \xrightarrow{a:1} q_0 \xrightarrow{b:2} q_1 \xrightarrow{c:3} q_1 \xrightarrow{b:2} q_1 \xrightarrow{b:2} q_1 \xrightarrow{c:3} q_1$, with output 1123223. The input a ends in q_0 (not accepting) and ba gets stuck, so both are rejected.

(c) **Regular grammars and FSAs.** Yes, they are equivalent. Here *equivalent* means they describe exactly the same class of languages, the regular languages. For every regular grammar there is an FSA accepting the same language, and for every FSA there is a regular grammar generating the same language. (Here “regular grammar” means right-linear, or consistently left-linear.) Regular expressions also describe the same class of languages.

(d) **Regular expressions and FSAs.** **Yes.** Every regular expression can be converted into an FSA. The algorithm (Thompson’s construction) is recursive:

- A single symbol x becomes two states joined by an arc labeled x .
- *Concatenation RS* : connect the accepting state of R to the start state of S with an ε -arc.
- *Union $R \mid S$* : add a new start state with ε -arcs into both machines.
- *Star R^** : add ε -arcs allowing the machine to be skipped or repeated.

Below is the resulting non-deterministic FSA for $(a^*bc^* \mid de)^*fg^*h$. I merged some redundant ε -arcs from the textbook construction to keep the picture readable. State L is the loop head, where each pass through the star starts.



How it works. From L , the machine either (i) takes the a^*bc^* branch ($A_0 \rightarrow A_1$), then returns to L with an ε -arc, (ii) takes the de branch (D_1), then returns to L , or (iii) leaves the star with f . Returning to L gives the outer $*$, and since L is the start state, zero iterations are allowed. After f , state F_1 loops on g (the g^*) and finally reads h to reach the accepting state F_2 . For example, $abdefgh$ is accepted, and fh is accepted with zero iterations of the star.